DiT Research

Aakash Agarwal

December 2025

1 Implementation

This assignment implements a Diffusion Transformer (DiT) architecture adapted for the Fashion-MNIST dataset. GitHub.[1]

1.1 Direct Patch Processing and Fixed Variance

Unlike standard Latent Diffusion Models, our implementation eliminates the pre-trained VAE stage. Instead, the Transformer operates directly on raw image patches:

- **Pixel-Space Patchify:** The 28×28 input images are directly divided into non-overlapping patches and linearly projected to the embedding dimension, treating raw pixel data as tokens.
- **Fixed Variance:** We employ a fixed linear variance schedule β_t rather than learning the diagonal covariance Σ_θ.
- **Noise Prediction:** The model is parameterized to predict the noise $\epsilon_\theta(x_t, t, y)$ added at timestep t. This prediction is used to derive the mean μ_θ of the posterior distribution for the reverse process, rather than predicting the mean directly.

1.2 Architectural Variations

We implemented and compared two distinct mechanisms for conditioning this noise prediction network:

DiT with In-Context Conditioning

This architecture is illustrated in Figure 1. Conditioning information is fused into a single token $c_{token} = \text{TimeEmb}(t) + \text{ClassEmb}(y)$ and prepended to the input sequence. This allows the mechanism to handle conditioning through standard self-attention:

$$\text{Sequence} = [c_{token}, x_1, x_2, \ldots, x_L] \tag{1}$$

DiT with adaLN-Zero

This architecture is illustrated in Figure 2. This variant uses Adaptive Layer Normalization (adaLN) to modulate the activation x based on the conditioning vector c, utilizing zero-initialized gating α for stability:

$$\text{adaLN}(x, c) = \gamma(c) \cdot \text{LayerNorm}(x) + \beta(c) \tag{2}$$

[1] `https://github.com/aakash-agarwal-002/diffusion-fashion-mnist`

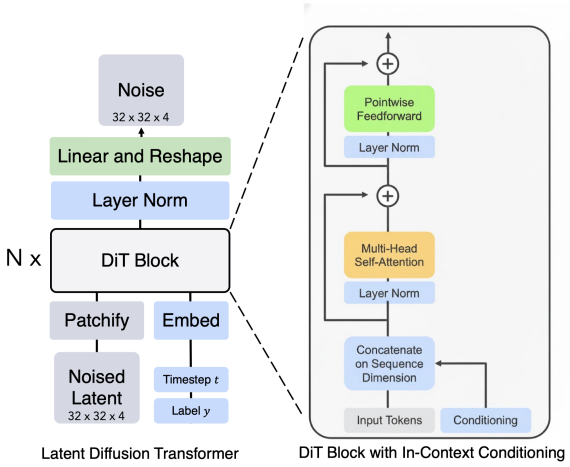

Figure 1: DiT with In-Context Conditioning

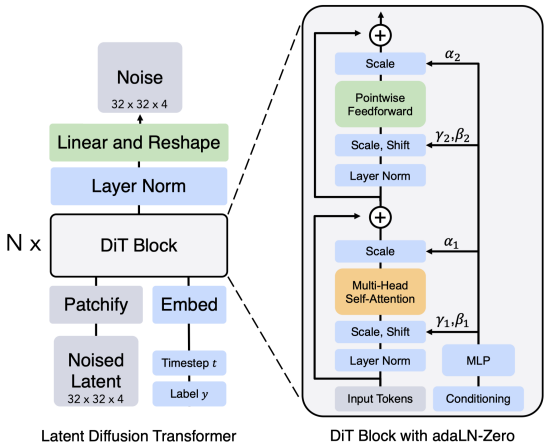

Figure 2: DiT with adaLN-Zero

2 Training and Inference

2.1 Implementation Details

The following parameters and configurations are shared across both Diffusion Transformer architectures implemented for this project.

2.1.1 Dataset and Preprocessing

- **Dataset:** The models are trained on the **FashionMNIST** dataset.
- **Resolution:** Input images are single-channel (grayscale) with a resolution of 28×28 pixels.
- **Normalization:** Pixel values are linearly scaled to the range $[0, 1]$ (or $[-1, 1]$ depending on the specific diffusion formulation).
- **Patchify Strategy:**
  - The 28×28 input image is divided into non-overlapping patches of size $P \times P$ where $P = 7$.
  - This results in a grid of size $(28/7) \times (28/7) = 4 \times 4$.
  - The total number of patch tokens is $L = 16$.
  - These patches are flattened and projected to a latent dimension $D = 256$.
- **Positional Embeddings:** Standard learnable positional embeddings are added to the patch tokens to retain spatial information. (actual paper uses standard ViT frequency-based positional embeddings (the sine-cosine version) to all input tokens.)

2.1.2 Diffusion Process (DDPM)

- **Framework:** Both implementations follow the Denoising Diffusion Probabilistic Model (DDPM) framework.
- **Noise Schedule:** A linear β-schedule is employed with the following parameters:
  - $\beta_{start} = 10^{-4}$
  - $\beta_{end} = 2 \times 10^{-2}$
  - Total Timesteps $T = 1000$
- **Forward Process:** Gaussian noise ϵ is added to the image x_0 to produce x_t according to the timestep t using the schedule $\bar{\alpha}_t$:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I}) \tag{3}$$

- **Reverse Process:** The generative process approximates the intractable posterior $q(x_{t-1}|x_t)$ as a Gaussian transition $p_\theta(x_{t-1}|x_t)$. The model is parameterized to predict the noise $\epsilon_\theta(x_t, t, y)$ contained within the latent x_t. This prediction is used to derive the mean of the posterior, allowing us to sample x_{t-1} via the stochastic update rule:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon_\theta(x_t, t, y)\right) + \sigma_t z \tag{4}$$

  where $z \sim \mathcal{N}(0, I)$ is standard Gaussian noise (added for $t > 1$) and α_t represents the parameters of the fixed variance schedule.

2.1.3 Training Configuration

- **Objective:** The models minimize the Mean Squared Error (MSE) between the actual added noise ϵ and the predicted noise ϵ_θ:

$$L = ||\epsilon - \epsilon_\theta(x_t, t, y)||^2 \tag{5}$$

- **Optimizer:** Adam optimizer with a learning rate of 10^{-3}.
- **Batch Size:** 128.
- **Epochs:** The model is trained for 25 epochs.
- **Evaluation Metric:** Fréchet Inception Distance (FID) using an InceptionV3 feature extractor (64-dimensional feature layer).
  - Comparison: 1,000 generated images vs. 1,000 real test images.
  - Evaluated at inference timesteps $T \in \{100, 500, 1000\}$.

2.2 Model-Specific Implementation Details

2.2.1 DiT with adaLN-Zero

This architecture utilizes Adaptive Layer Normalization to inject conditioning information directly into the normalization layers.

- **Backbone:** Vision Transformer (ViT) with **Adaptive Layer Normalization (adaLN)**.
- **Architecture Depth:** 6 Transformer Blocks.
- **Conditioning Mechanism (adaLN-Zero):**
  - A conditioning vector c is derived from the sum of time and class embeddings:

$$c = \text{TimeEmb}(t) + \text{ClassEmb}(y) \tag{6}$$

  - This vector is passed into an MLP layer (with SiLU activation) to regress scale (γ), shift (β), and gating factors (α) for the residual connections.
  - The normalization is applied as:

$$\text{adaLN}(x, c) = \gamma(c) \cdot \text{LayerNorm}(x) + \beta(c) \tag{7}$$

- **Output Layer:** Output tokens are projected back to pixel space, reshaped to 28×28.

2.2.2 DiT with In-Context Conditioning

This architecture treats conditioning information as an additional token in the sequence, similar to a BERT [CLS] token.

- **Backbone:** Vision Transformer (ViT) with **Input Token Conditioning**.
- **Architecture Depth:** 6 Transformer Blocks.
- **Hidden Dimension:** 1024 (MLP expansion ratio of 4).
- **Conditioning Mechanism (Token Prepending):**
  - **Fusion:** Time and class embeddings are summed to create a single conditional token:

$$c_{token} = \text{TimeEmb}(t) + \text{ClassEmb}(y) \tag{8}$$

  - **Prepending:** This token is prepended to the patch sequence, extending length from 16 to 17:

$$\text{Input Sequence} = [c_{token}, x_1, x_2, \ldots, x_{16}] \tag{9}$$

- **Normalization:** Uses Standard LayerNorm (Pre-Norm formulation).
- **Output Head:** The conditional token c_{token} is discarded after the transformer layers. The remaining tokens are projected back to pixel space.

2.3 Loss

To optimize the Diffusion Transformer, we utilized the Mean Squared Error (MSE) loss function, which quantifies the discrepancy between the actual Gaussian noise ϵ added to the latent and the noise ϵ_θ predicted by the model. The training objective is formally defined as $L = ||\epsilon - \epsilon_\theta(x_t, t, y)||^2$. The models were trained using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 128 over 25 epochs.

The training progression is visualized in Figure 3 and Figure 4. Both the In-Context Conditioning and adaLN-Zero architectures demonstrate stable convergence behaviors. As observed in the plots, the loss values exhibit a sharp descent during the initial epochs, followed by a gradual stabilization, indicating that the models effectively learned to approximate the noise distribution within the allotted training duration.

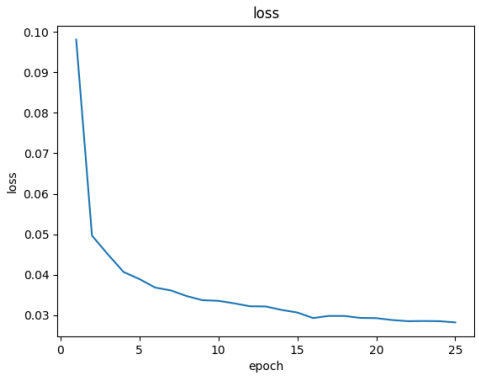

Figure 3: Context Conditioning loss epoch 25

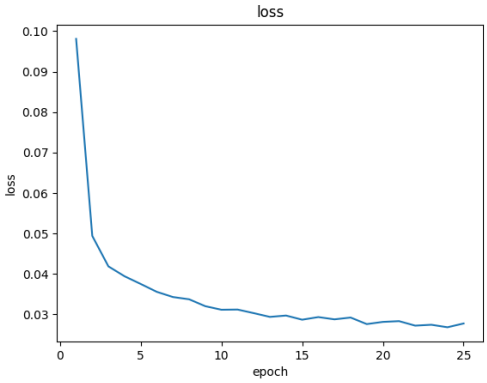

Figure 4: adaLN Zero loss epoch 25

2.4 Evaluation

The model evaluation strategy incorporates both qualitative visual inspection to assess generation fidelity and quantitative benchmarking to measure distribution similarity.

2.4.1 Qualitative Analysis (Visual Sampling)

Visual evaluation is performed periodically during training to monitor the model's progress and the quality of conditional generation.

- **Sampling Frequency:** Generated samples are saved every 5 epochs.
- **Class Coverage:** A fixed set of 10 samples is generated for each evaluation step, corresponding to exactly one image per class (0–9) from the FashionMNIST dataset.
- **Inference Timesteps:** To evaluate the trade-off between sampling speed and image quality, visual samples are generated using three distinct total inference timesteps: $T \in \{100, 500, 1000\}$.
- **Post-Processing:** Generated latent tensors are clamped to the range $[0, 1]$ before being saved as images.

2.4.2 Quantitative Analysis (Fréchet Inception Distance)

The Fréchet Inception Distance (FID) is utilized to quantify the distance between the distribution of real images and generated images. Lower FID scores indicate higher image quality and diversity.

- **Implementation:** The metric is computed using the `torchmetrics` library.
- **Feature Extractor:** The standard InceptionV3 network is used as the feature extractor, specifically utilizing the 64-dimensional feature layer (`feature=64`) rather than the standard 2048-dimensional layer.
- **Sample Size:**

  - **Real Distribution:** 1,000 images sampled from the FashionMNIST test set.
  - **Fake Distribution:** 1,000 images generated by the model using random noise and random class labels.

- **Data Formatting:**
  - Since FashionMNIST images are grayscale (1 channel) and InceptionV3 expects RGB input, all images are converted to pseudo-RGB by repeating the single channel 3 times.
  - Images are cast to `uint8` format (0–255 range) prior to feature extraction.

- **Performance Sweeps:** FID scores are calculated at the end of training using the best checkpoint. The evaluation is repeated for varying inference timesteps ($T \in \{100, 500, 1000\}$) to analyze the impact of the diffusion schedule length on generation quality.

2.5 Results

2.5.1 Qualitative Analysis

The visual progression of generated samples for both architectures is presented in Figure 5 (In-Context Conditioning) and Figure 6 (adaLN-Zero).

- **Training Progression:** Both models demonstrate a clear improvement in generation quality over the course of 25 epochs. Early epochs (e.g., Epoch 5) exhibit recognizable shapes but lack fine-grained texture, while later epochs (Epoch 20-25) produce sharp, distinct fashion items.

- **Impact of Inference Timesteps (T):** There is a significant correlation between the number of inference timesteps and image fidelity.
  - $T = 100$**:** Samples generated with only 100 steps (top rows of Figures 5 & 6) appear noisier and less defined, retaining some artifacts of the initial Gaussian noise.
  - $T = 1000$**:** Samples generated with the full 1000 steps (bottom rows) exhibit the highest visual quality with sharp boundaries and correct class-specific features.

- **Class Conditioning:** Qualitatively, both mechanisms successfully incorporate class labels.The columns in the generated grids consistently correspond to their target classes (e.g., the "Trouser" column consistently contains trousers), confirming that both Token Prepending and adaLN-Zero effectively guide the diffusion process toward the conditional distribution.

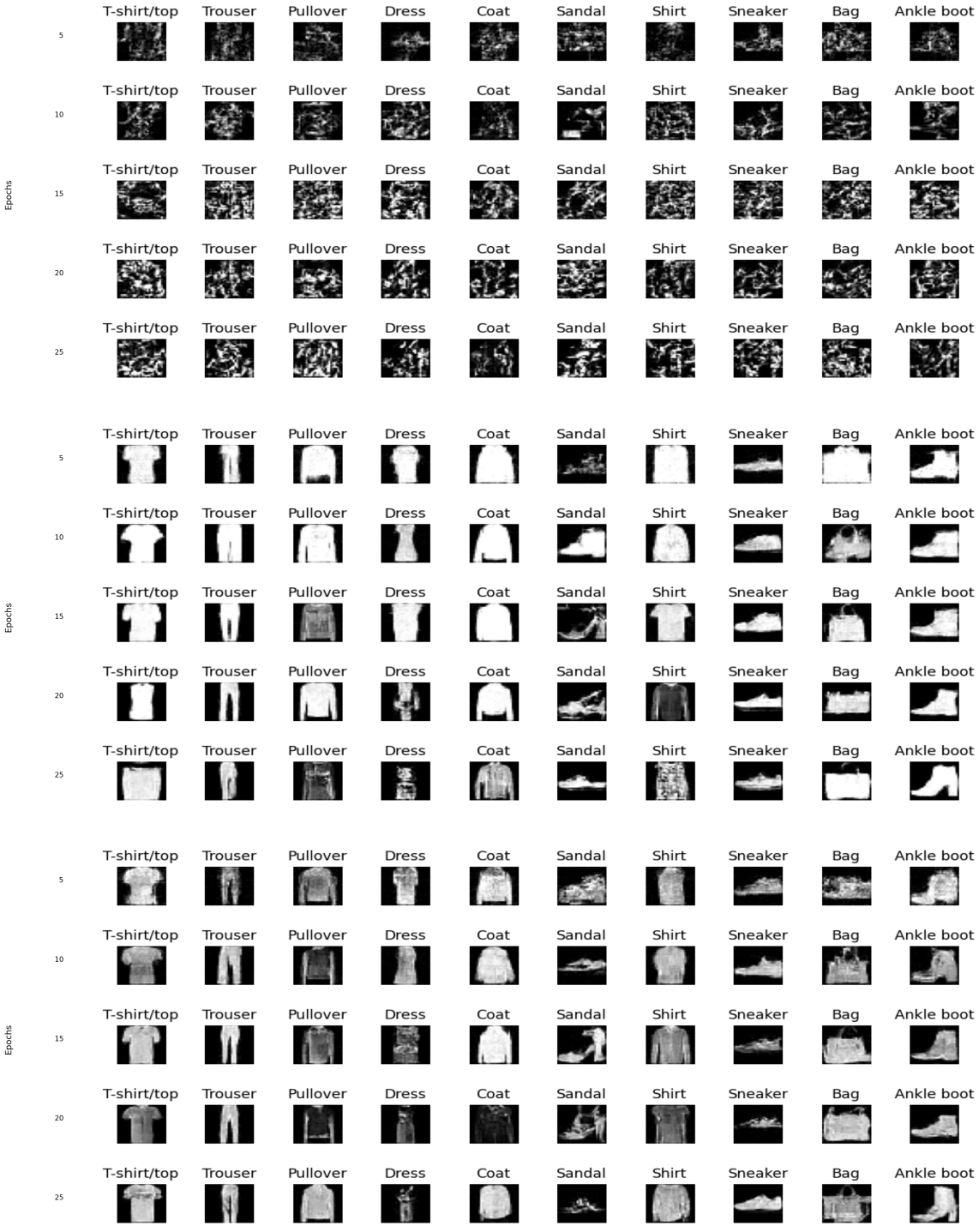

Figure 5: Results for In Context Conditioning for epochs 5 to 25 for three different total inference timesteps 100 (top) 500 and 1000 (bottom).

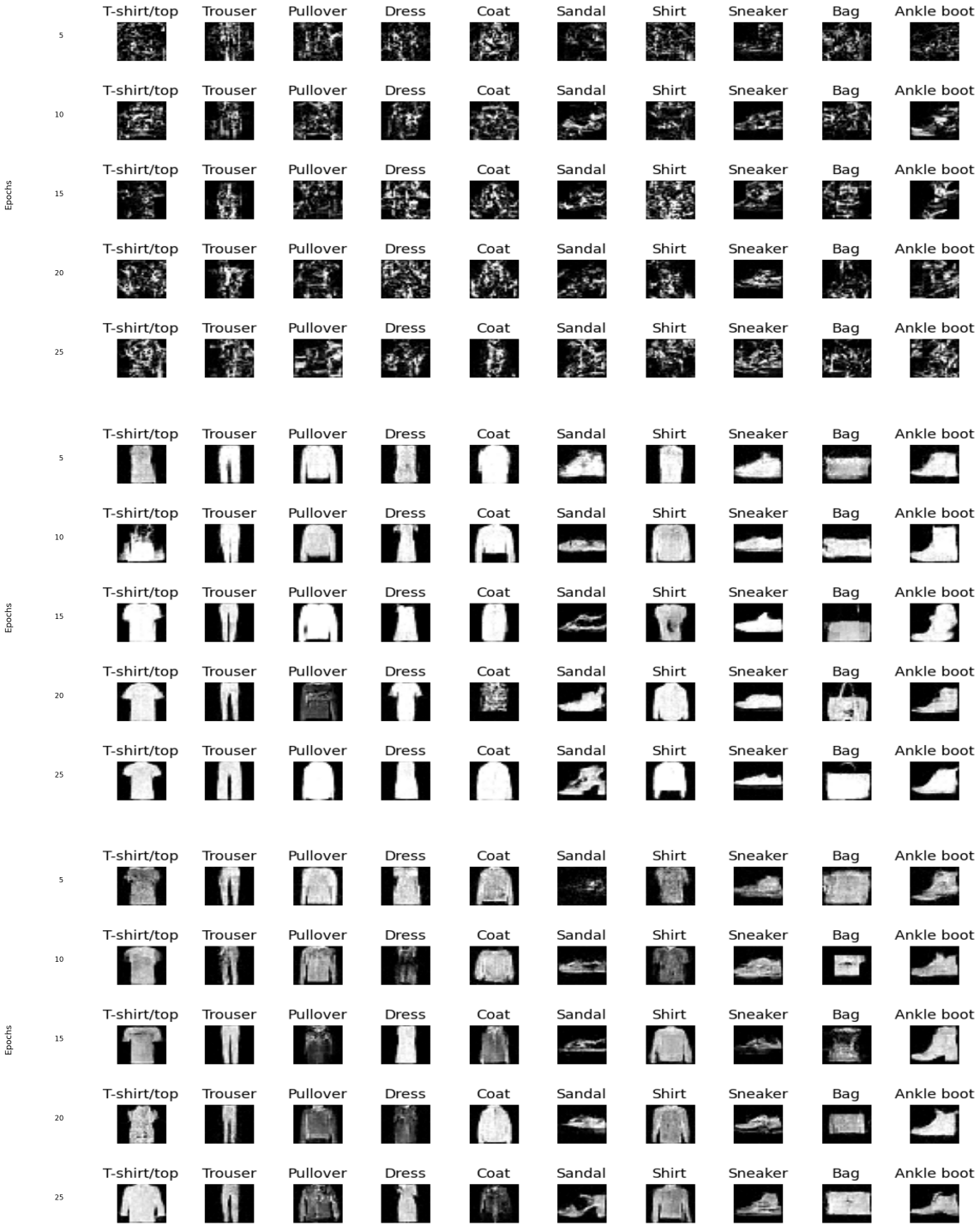

Figure 6: Results for adaLN Zero for epochs 5 to 25 for three different total inference timesteps 100 (top), 500 and 1000 (bottom).

2.5.2 Quantitative Analysis (Fréchet Inception Distance)

To quantitatively benchmark the performance, we calculated the Fréchet Inception Distance (FID) scores across three different inference timestep settings ($T \in \{100, 500, 1000\}$). Lower FID scores indicate a distribution closer to the real FashionMNIST test set.

The comparative results are summarized in Table 1.

Table 1: Quantitative Comparison of FID Scores across different inference timesteps (T). Lower is better.

Model Variant	$T = 100$	$T = 500$	$T = 1000$
In-Context Conditioning	3.362	1.156	0.065
adaLN-Zero	**2.587**	**1.029**	**0.054**

Observations

1. **Correlation with Sampling Steps:** Consistent with the qualitative results, the FID score decreases monotonically as the number of inference steps increases. For the In-Context model, increasing steps from 100 to 1000 results in a massive performance gain, dropping the FID from approximately 3.36 to 0.065.
2. **Model Comparison:** The **adaLN-Zero** architecture consistently outperforms the In-Context Conditioning approach across all timestep settings.
   - At $T = 1000$, adaLN-Zero achieves an extremely low FID of **0.0541**, compared to **0.0654** for In-Context Conditioning.
   - The performance gap is most pronounced at lower distinct timesteps ($T = 100$), where adaLN-Zero leads by approximately 0.77 points (2.587 vs 3.362).
3. **Conclusion:** While both models converge to excellent results given sufficient sampling steps, the adaLN-Zero mechanism—which modulates the normalization layers directly—appears to provide a more efficient and stable path to the target distribution than simple token prepending.

3 Ablation Study

To quantify the specific contributions of the architectural innovations implemented in Section 2, we performed an ablation study by systematically removing key components from both the **adaLN-Zero** and **In-Context Conditioning** models. This analysis aims to verify whether the complexity added by these mechanisms translates to tangible performance gains

3.1 Impact of Gating in adaLN-Zero (Removal of α)

Motivation The standard adaLN-Zero implementation initializes the residual blocks as identity functions by regressing a gating parameter α initialized to zero. This allows the model to act as a standard VAE initially and progressively learn the diffusion mapping. We aimed to test if this zero-initialization gating is strictly necessary for convergence on simple datasets like FashionMNIST.

Methodology A visual comparison of both variants is provided in Figure 7. We modified the MLP regressor in the adaLN-Zero block. In the baseline implementation, the MLP regresses three parameters: scale (γ), shift (β), and the gate (α).

- **Baseline:** $Block(x) = x + \alpha \cdot \text{Function}(x, c)$
- **Ablated Model:** We removed the α parameter entirely. The MLP only regresses γ and β. The residual connection treats the block output with a fixed weight of 1 (standard ResNet-style addition).

Hypothesis Removing α removes the "zero-init" property. We hypothesize this may lead to training instability in early epochs or higher final validation loss, as the model cannot easily default to an identity mapping at initialization.

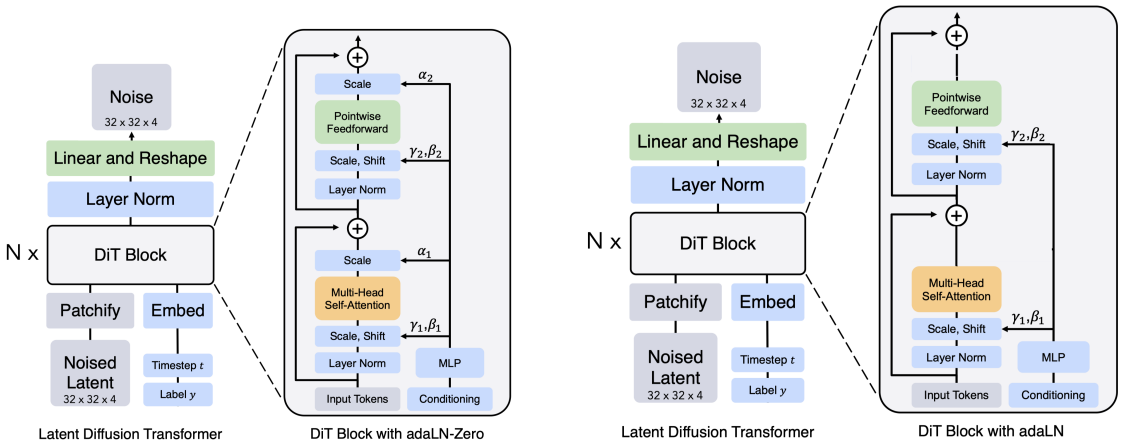

Figure 7: adaLN Zero (left) and the corresponding adaLN variant obtained by removing the parameter α (right)

3.2 Impact of Class Conditioning in Token Prepending

Motivation The baseline In-Context architecture fuses time and class information into a single token: $c_{token} = TimeEmb(t) + ClassEmb(y)$. We sought to isolate the effect of class labels to determine if the model could effectively learn the image distribution using only timestep information (unconditional generation).

Methodology The resulting architecture used for this ablation is illustrated in Figure 8. We modified the conditioning token to exclude the class embedding, making the generation process unconditional regarding the object type.

- **Baseline:** The conditioning token contains summed embeddings: $c_{token} = TimeEmb(t) + ClassEmb(y)$.

- **Ablated Model:** The class embedding $ClassEmb(y)$ was removed. The conditioning token relies solely on the timestep:

$$c_{token} = TimeEmb(t) \tag{10}$$

The input sequence length remains $L+1$, but the prepended token now carries only temporal information.

Hypothesis Since the timestep t is preserved, the model should still be able to learn the diffusion noise schedule and generate realistic images. However, we hypothesize that the model will lose all ability to adhere to specific class prompts (e.g., generating a specific fashion item when requested), effectively functioning as an unconditional probabilistic generator.

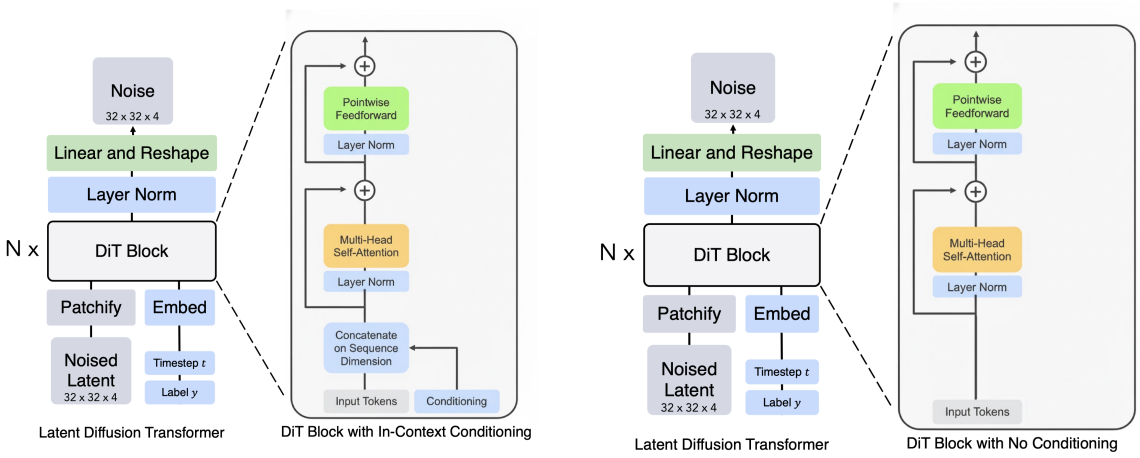

Figure 8: In context conditioning (left) compared to the no conditioning baseline (right)

3.3 Comparative Results

To evaluate the impact of these ablations, we compared the ablated models against the baselines trained in Section 2 using the same hyperparameters (25 epochs, lr=10^{-3}).

Table 2: Ablation Study Results: Comparison of Baseline vs. Ablated Architectures across Inference Timesteps

			FID Score ↓		
Model Variant	**Component Removed**	**Observations**	**T=100**	**T=500**	**T=1000**
adaLN-Zero (Base)	None	Stable generation, clear class separation	2.587	1.029	**0.054**
adaLN-Zero (Ablated)	Gating (α)	Slower convergence, higher noise levels	2.922	1.468	0.113
In-Context (Base)	None	Accurate class control (matches label)	3.362	1.156	0.065
In-Context (Ablated)	Class Label (y)	High quality images, but random classes	4.367	**0.518**	0.086

Analysis For **adaLN-Zero**, the removal of α demonstrates the importance of the zero-initialization for stabilizing the deep transformer depth. For the **In-Context** model, the ablation confirms that the "Prepend" strategy successfully carries timestep information even without class labels. The model converged to generating valid FashionMNIST samples, proving that the architecture functions well as an unconditional generator, though it ignored the input labels y during inference.

4 Inference Optimization

4.1 Strategies for Inference Optimization in Diffusion Transformers

The iterative nature of the reverse diffusion process, typically requiring $T = 1000$ sequential function evaluations, presents a significant bottleneck for real-time applications. To address this, the research community has developed various strategies to accelerate inference without compromising generation quality. These can be broadly categorized into three approaches:

1. **Training-Free Numerical Solvers:** These methods reformulate the diffusion process as a differential equation (ODE or SDE) and utilize higher-order numerical solvers to reduce the number of sampling steps.
    - **DDIM (Denoising Diffusion Implicit Models):** Reformulates the stochastic process as a deterministic non-Markovian trajectory, allowing for "strided" sampling (skipping steps).
    - **DPM-Solver & UniPC:** Employ high-order Taylor expansion methods to solve the probability flow ODE, enabling high-quality generation in as few as 10-20 steps.
2. **Knowledge Distillation:** These techniques require re-training or fine-tuning to "distill" the knowledge of a slow teacher model into a faster student model.
    - **Progressive Distillation:** Halves the number of required steps iteratively, ultimately achieving 4-8 step sampling.
    - **Consistency Models:** Train models to map any point on the trajectory directly to the origin x_0, enabling 1-step or few-step generation.

3. **System-Level Optimizations:** Focuses on reducing the latency of each individual forward pass rather than the total number of steps.
    - **Attention Optimization:** Techniques like FlashAttention and Memory-Efficient Attention reduce the quadratic complexity of the Transformer's self-attention mechanism.
    - **Quantization:** Reducing model precision from FP32 to FP16 or INT8 to accelerate matrix multiplications and reduce memory bandwidth usage.

Our Approach: While distillation and system-level optimizations offer significant speedups, they often require complex re-training pipelines or hardware-specific tuning. In this work, we focus on **Algorithmic Optimization** via **DDIM**. This allows us to accelerate our custom Diffusion Transformer (DiT) purely through inference-time changes, maintaining the flexibility of the original trained weights while reducing the sampling steps from 1000 to roughly 50-100.

4.2 Methodology: From DDPM to DDIM

While the Denoising Diffusion Probabilistic Models (DDPM) framework yields high-quality samples, the reverse diffusion process is inherently slow due to its Markovian nature, requiring T sequential forward passes (typically $T = 1000$) to generate a single image. To address this bottleneck, we implemented Denoising Diffusion Implicit Models (DDIM) to accelerate sampling without the need for model retraining.

In our implementation, we replaced the stochastic DDPM sampler with a deterministic DDIM sampler. The DDPM reverse step:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t) \right) + \sigma_t z \tag{11}$$

is replaced by the non-Markovian deterministic update rule ($\sigma_t = 0$):

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{1-\bar{\alpha}_t} \cdot \epsilon_\theta(x_t)}{\sqrt{\bar{\alpha}_t}} \right) + \sqrt{1-\bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t) \tag{12}$$

This formulation allows for "strided" sampling, where we skip intermediate timesteps. We introduced a sampling interval parameter K (denoted as `interval` in the code), such that the inference process iterates through a subset of timesteps $\{T, T-K, T-2K, \ldots, 0\}$. This reduces the total number of function evaluations from T to $\lceil T/K \rceil$.

4.3 Quantitative Analysis

We evaluated the trade-off between inference speed and generation quality (measured by Fréchet Inception Distance, FID) across varying timesteps ($T \in \{100, 500, 1000\}$) and skip intervals ($K \in \{1, 2, 5, 10, 20\}$). The experiments were conducted on the FashionMNIST dataset.

4.4 Results Discussion

The results in Table 3 demonstrate significant efficiency gains with DDIM. Notably, using DDIM with $T = 1000$ and a skip interval of $K = 10$ (100 effective steps) reduced inference time to

Table 3: Comparison of Inference Performance: DDPM vs. DDIM. Time is measured in seconds per batch. Lower FID indicates better image quality.

Method	**Timesteps (T)**	**Interval (K)**	**Effective Steps**	**Time (s)**	**FID ↓**
Baseline: DDPM (Stochastic)					
DDPM	100	1	100	50.00	2.5860
DDPM	500	1	500	180.00	1.0286
DDPM	1000	1	1000	216.00	**0.0541**
Optimization: DDIM (Deterministic)					
DDIM	100	1	100	0.4043	2.5568
DDIM	100	2	50	0.1843	2.5173
DDIM	100	5	20	0.0626	2.4613
DDIM	100	10	10	0.0282	2.3675
DDIM	100	20	5	**0.0159**	2.3461
DDIM	500	1	500	2.1148	0.2457
DDIM	500	2	250	1.0617	0.2511
DDIM	500	5	100	0.4006	0.2635
DDIM	500	10	50	0.1894	0.2818
DDIM	500	20	25	0.0831	0.2874
DDIM	1000	1	1000	4.2589	0.2072
DDIM	1000	2	500	2.1346	0.2086
DDIM	1000	5	200	0.8220	0.2137
DDIM	1000	10	100	0.3984	**0.1640**
DDIM	1000	20	50	0.1887	0.1570

0.3984 seconds—a speedup of over 500× compared to the DDPM baseline (216 seconds)—while maintaining a highly competitive FID score of 0.1640.

We observed that the DDIM sampler is more robust to step reduction than DDPM. Even with aggressive skipping (e.g., $T = 1000, K = 20$, resulting in only 50 steps), the model achieved an FID of 0.1570, suggesting that the deterministic trajectory is easier to approximate with fewer steps than the stochastic random walk of DDPM.

Qualitative results for this aggressive optimization strategy are visualized in Figure 9. Despite the reduction to only 50 effective sampling steps, the model maintains high generation fidelity and clear class separation. Furthermore, Figure 10 illustrates the stability of the method across varying skip intervals ($K \in \{1, 2, 5, 10, 20\}$). As observed in these samples, the image quality does not degrade significantly even as the interval increases, confirming the robustness of the deterministic update rule.

Figure 9: Inference optimization for 20 iterval steps at $T = 1000$

Figure 10: Inference optimization at epoch 25 with interval $\{1,2,5,10,20\}$ evaluation at $T = 1000$